

MATH3881/5881: Statistical Machine Learning Theory

Week 8: Foundations of Sampling & Generative Modelling

Sahani Pathiraja

UNSW, Sydney

Key Topics

8.1	Sampling vs Generative Modelling	8-2
8.2	Classical sampling methods	8-3
8.3	Statistical Distances	8-7
8.4	Recap: Maximum Likelihood Estimation	8-11
8.5	Latent Variable modelling	8-15
8.6	Variational Inference (VI)	8-20
8.7	Tutorial exercises	8-28
8.8	Supplementary Material [NON-EXAMINABLE]	8-31
References		8-31

8.1 Sampling vs Generative Modelling

The remaining lectures of the course will change in flavour from the primarily deterministic approaches we've seen up until now. Previously, we focused on fitting a single model to a dataset, i.e. a neural network with a single set of parameters. This model then generates a single set of outputs for any given input. We will now focus on methods that can produce multiple plausible outputs for a single input. A classical example of this in generative modelling is to produce multiple plausible images of animals. Every time the model is queried, it can produce a different image of an animal. Another example you would have all come across is in large language models; many of these produce slightly different responses even when queried with exactly the same prompt each time.

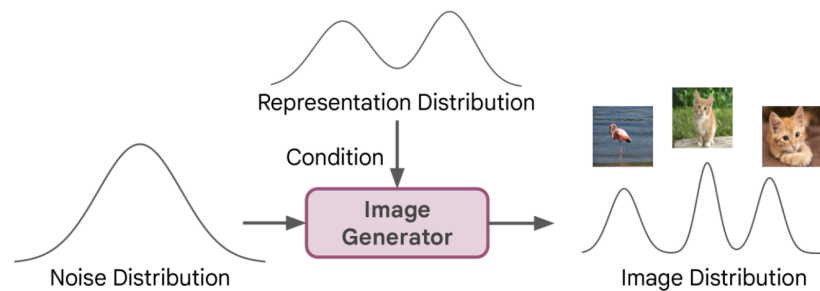


Figure 8.1: Schematic of generating images of animals, from [1]

The relevant topics here are *sampling* and *generative modelling*. In the machine learning context, the term sampling usually refers to generating realisations or samples from a known distribution using some computational method.¹ The main difference between the sampling and generative modelling is that in sampling, you have access to the desired probability density function or mass function (up to a normalisation constant) that you would like to generate a sample from. In generative modelling, you instead want to generate a new sample, given an existing sample. In other words, you only have access to the *empirical distribution* (see Section 8.2.2), so that you need a different set of techniques than those used in sampling.

At a more fundamental level, generative modelling involves learning the overall data generating process for all variables of interest. In fact you have likely already seen a form of generative modelling before in classical statistics: e.g. given data on the number of heads in an experiment with p coin flips, you choose a binomial probability model and fit the success probability using the observed number of heads using maximum likelihood estimation. This yields a data generating process (the binomial with known success probability) that you can use to gain further insights or simulate future outcomes. One of the most distinctive features of modern generative modelling is in its *flexibility*. It is tailored to (typically) large datasets where minimal assumptions are made by the modeller on the form of the data generating process through the use of neural network based parameterisations. This is not to say that no choices are made; certainly the choice of the neural network architecture has an influence, but the goal is to let the data speak for itself rather than

¹not to be confused with sampling theory in frequentist statistics which is focused on confidence intervals, p-values etc.

relying on a classical parametric statistical model with fixed properties. There is always a place for both types of modelling (certainly, one does not need a complex neural network to model coin tosses).

In the rest of this lecture, we'll review some of the important foundational elements from probability and statistics that have influenced modern generative modelling methods.

8.2 Classical sampling methods

Given a continuous random variable $X \in \mathbb{R}^d$ with distribution described by a potentially unnormalised probability density function $p(x)$, $x \in \mathbb{R}^d$, how can we generate a sample $\{x_1, x_2, \dots, x_N\}$ where $x_i \sim p(x)$ for $i \in \{1, 2, \dots, N\}$? This is the central question of *sampling*, and has mainly found applications in Bayesian inference, statistical physics, molecular dynamics etc. There are an endless pool of methods for this task, all with their advantages and drawbacks, including rejection sampling, importance sampling, sequential importance sampling, Markov Chain Monte Carlo etc. (see MATH3871/5960 Bayesian Inference and Computation for more!). Perhaps one of the simplest approaches is inverse transform sampling. Before describing this, we'll briefly recap the change of variable technique.

8.2.1 Change of variable formula

It is often necessary to find the distribution of a random variable Y that is defined as a function g of another random variable X (i.e., $Y = g(X)$). For continuous random variables, this relationship is governed by the change-of-variable formula, which we recap below.

Change of variable formula

For a multivariate continuous random variable $X \in \mathbb{R}^d$ and an invertible, differentiable function $g : \mathbb{R}^d \rightarrow \mathbb{R}^d$, the probability density function (pdf) of $Y = g(X)$ is given by

$$p_Y(y) = p_X(g^{-1}(y)) \cdot \left| \det \left(\frac{\partial}{\partial y} g^{-1}(y) \right) \right| \quad (8.1)$$

where $\frac{\partial}{\partial y} g^{-1}(y)$ is the Jacobian matrix. The term $\left| \det \left(\frac{\partial}{\partial y} g^{-1}(y) \right) \right|$ acts as a scaling factor. It measures how much a “unit volume” changes when applying the transformation g .

The change of variable formula turns out to be very useful in sampling and generative modelling methods² and variational inference type methods (more later). In the following example, we see how it can be used to generate two correlated Gaussian random variables from two independent univariate Gaussian samples.

²e.g. in normalising flows, not covered in this course

Example: Generating bivariate Gaussian samples

Suppose that we can generate univariate independent standard Gaussians, $Z_1, Z_2 \stackrel{\text{i.i.d.}}{\sim} N(0, 1)$ and let $\mathbf{Z} = \begin{bmatrix} Z_1 \\ Z_2 \end{bmatrix}$. Their joint density is given by

$$f_{\mathbf{Z}}(z_1, z_2) = \frac{1}{2\pi} \exp\left(-\frac{z_1^2 + z_2^2}{2}\right).$$

Let $\rho \in (-1, 1)$ and define

$$\mathbf{X} = \begin{bmatrix} X \\ Y \end{bmatrix} = A \begin{bmatrix} Z_1 \\ Z_2 \end{bmatrix}, \quad A = \begin{pmatrix} 1 & 0 \\ \rho & \sqrt{1 - \rho^2} \end{pmatrix}.$$

This corresponds to an invertible linear transformation, with inverse

$$A^{-1} = \begin{bmatrix} 1 & 0 \\ -\frac{\rho}{\sqrt{1 - \rho^2}} & \frac{1}{\sqrt{1 - \rho^2}} \end{bmatrix}$$

and $\det(A) = \sqrt{1 - \rho^2}$. The multivariate change-of-variables formula (8.1) gives

$$\begin{aligned} f_{\mathbf{X}}(x, y) &= f_{\mathbf{Z}}\left(x, \frac{y - \rho x}{\sqrt{1 - \rho^2}}\right) |\det(A^{-1})| \\ &= \frac{1}{2\pi\sqrt{1 - \rho^2}} \exp\left[-\frac{1}{2} \left(x^2 + \frac{(y - \rho x)^2}{1 - \rho^2}\right)\right] \\ &= \frac{1}{2\pi\sqrt{1 - \rho^2}} \exp\left(-\frac{x^2 - 2\rho xy + y^2}{2(1 - \rho^2)}\right), \end{aligned}$$

which is precisely the density of a bivariate Gaussian distribution with mean zero and covariance and covariance matrix

$$\Sigma = \begin{bmatrix} 1 & \rho \\ \rho & 1 \end{bmatrix}.$$

This is actually a special case of the standard procedure used in various programming languages to generate multivariate Gaussian random samples (see Section 8.8.1.)

8.2.2 Monte Carlo integration and Bayesian sampling

One of the main applications of modern sampling methods is in Bayesian inference. We will provide a very brief overview of the main concepts in Bayesian inference that will be needed for the remaining lectures. Loosely speaking, Bayesian inference aims to combine prior knowledge with observed data to obtain estimates of an unknown variable. These unknowns may be parameters of a data generating process, latent features or variables (see Section 8.5) or simply events of

interest. To make this problem concrete, consider the case of inference about a parameter θ of a data generating process $p_\theta(x)$, which we will write as $p(x|\theta)$ to make clear the connections to conditional probability. Bayes theorem says

$$p(\theta|x) = \frac{p(x|\theta)p(\theta)}{p(x)} \quad (8.2)$$

where

- $p(\theta)$ is the prior distribution (quantifying knowledge available before the experiment is conducted)
- $p(x|\theta)$ (sometimes written $L(\theta; x)$) is the likelihood of observing the data x given the assumed data generating process $p_\theta(x)$
- $p(x)$ is the model evidence.
- $p(\theta | x)$ is the posterior distribution, which quantifies our uncertainty about θ after updating our prior belief based on the observed data x

Although this formula is conceptually simple to write, its computation is challenging beyond a few parametric distributions³. Furthermore, one is often interested in generating samples from the posterior distribution in order to quantify uncertainty in predictions. For these reasons, there was an explosion of activity around sampling strategies in the Bayesian inference field, with Markov Chain Monte Carlo being the most famous suite of techniques (see MATH3871/5960 for more details).

One of the most fundamental enablers of computational Bayesian inference is *Monte Carlo integration*. It has enabled many complex high dimensional problems to be solved numerically. Monte Carlo integration is a numerical method for approximating integrals using random samples. It is particularly useful in high-dimensional problems where classical grid based methods become computationally infeasible.

Monte Carlo estimator

Suppose we want to compute an integral of the form

$$\mathcal{I} = \int_D f(x) p(x) dx = \mathbb{E}_p[f]$$

where $p(x)$ is a probability density function on D , the support, e.g. $D = \mathbb{R}^d$. The Monte Carlo estimator of I is

$$\hat{\mathcal{I}}_N = \frac{1}{N} \sum_{i=1}^N f(X_i); \quad X_i \sim p(x)$$

where $X_1, X_2, \dots, X_N \sim p$ are iid.

³for further reading, see Conjugate priors.

The Monte Carlo Estimator is powerful because of its simplicity, and also because it is an unbiased estimator and its variance is easily quantifiable as in the following result. The mean squared error (MSE) of an estimator $\hat{\theta}$ (where θ usually refers to the parameter of a statistical model) is

$$\text{MSE}(\hat{\theta}) = \mathbb{E} \left[(\hat{\theta} - \theta)^2 \right] = \text{Var}(\hat{\theta}) + \text{Bias}(\hat{\theta})^2.$$

where $\text{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta$.

Proposition 8.2.1

Let $X_1, \dots, X_N$ be independent and identically distributed according to p , and define $I = \mathbb{E}_p[f(X)]$. Assume that $\mathbb{E}_p[|f(X)|] < \infty$, then the Monte Carlo estimator $\hat{I}_N = \frac{1}{N} \sum_{i=1}^N f(X_i)$ is unbiased, i.e.

$$\mathbb{E}[\hat{I}_N] = I.$$

Furthermore, if $\mathbb{E}_p[f(X)^2] < \infty$, then

$$\text{Var}(\hat{I}_N) = \frac{\text{Var}(f(X))}{N} = \text{MSE}(\hat{I}_N).$$

Proof: For unbiasedness, first note that since X_i are identically distributed, $\mathbb{E}[f(X_i)] = I$ for all i . Then by the linearity of expectation and the fact that

$$\mathbb{E}[\hat{I}_N] = \mathbb{E} \left[\frac{1}{N} \sum_{i=1}^N f(X_i) \right] = \frac{1}{N} \sum_{i=1}^N \mathbb{E}[f(X_i)] = \frac{1}{N} \sum_{i=1}^N I = \frac{NI}{N} = I.$$

For the variance, recall the variance property for a constant a and scalar random variable Y ,

$$\text{Var}(aY) = a^2 \text{Var}(Y).$$

Hence

$$\text{Var}(\hat{I}_N) = \frac{1}{N^2} \text{Var} \left(\sum_{i=1}^N f(X_i) \right) = \frac{1}{N^2} \sum_{i=1}^N \text{Var}(f(X_i)) = \frac{1}{N} \text{Var}(f(X))$$

where in the second equality we have used the independence of X_i for all i and the third equality we have used that X_i are identically distributed. ■

Remark 8.2.1

Proposition 8.2.2 is the justification for the widely stated claim that the (standard) error of a Monte Carlo estimator decreases at rate $O(N^{-\frac{1}{2}})$. This convergence rate is independent of dimension of x , which is one of the major advantages of Monte Carlo methods in high-dimensional integration. However, the dimension dependence usually appears in the “constant”, $\text{Var}(f(X))$. This is often not the subject of attention because it is problem-dependent, but is definitely important in the actual performance of a Monte Carlo method.

There are several advanced Monte Carlo methods that have been developed to improve on this rate, these are beyond the scope of this course.

Aside from high dimensional integrals, Monte Carlo methods usually involve estimating distributions by their *empirical distribution*. That is, suppose you have $X_i \sim p(x)$ for $i = 1, 2, \dots, N$ iid samples with x_i denoting the realised value. The empirical distribution $\hat{p}(x)$ approximates $p(x)$ by placing probability $1/N$ on each sample and is written as

$$\hat{p}(x) := \frac{1}{N} \sum_{i=1}^n \delta_{X_i}(x),$$

where δ_{x_i} denotes a Dirac point mass located at x_i . The Dirac delta is characterised informally by

$$\delta_{x_i}(x) = 0 \quad \text{for } x \neq x_i,$$

and by the integration identity for suitable functions f ,

$$\int f(x) \delta_{x_i}(x) dx = f(x_i)$$

so then

$$\int f(x) \hat{p}(x) dx = \frac{1}{n} \sum_{i=1}^n f(x_i).$$

8.3 Statistical Distances

At the heart of many sampling and generative modelling methods is some measure of “distance” between probability distributions. Just as there are different ways of measuring distance between two places (straight line distance, driving distance, walking distance), there are also different ways of quantifying the *discrepancy* between two probability distributions. As you will see in the next lectures on Variational Auto Encoders (VAEs) and Diffusion generative models, statistical distances are used as objective functions to fit neural networks via our familiar optimisation tools such as stochastic gradient descent. The statistical distance is the quantitative measure that answers the question: *If the data is generated from a distribution/data generating process $p(x)$, what is the mismatch if modelling it using distribution q ?* The term *statistical distance* is an umbrella term used to refer to probability metrics and also statistical divergences. They are defined below.

Probability metric

A true probability metric $d(p, q)$ must satisfy the following properties, for all distributions p, q, r ,

- $d(p, q) \geq 0$ (non-negativity)
- $d(p, q) = 0 \iff p = q$ (indiscernibility)
- $d(p, q) = d(q, p)$ (symmetry)
- $d(p, r) \leq d(p, q) + d(q, r)$ (triangle inequality)

Many important statistical distances are not metrics but *divergences*.

Divergences

A divergence $D(p\|q)$ between distributions p and q satisfies non-negativity and indiscernibility i.e.

$$D(p\|q) \geq 0, \quad D(p\|q) = 0 \iff p = q,$$

but need not be symmetric and need not obey the triangle inequality.

8.3.1 Kullback Leibler Divergence

One of the most important statistical distances that you will encounter in probabilistic machine learning is the Kullback-Leibler Divergence D_{KL} . As its name suggests, it is a divergence and not a probability metric. It measures the average log difference between two probability densities (for continuous random variables).

Kullback Leibler divergence

For probability density functions $p(x)$ and $q(x)$ describing continuous random variables in $\mathbb{R}^d$ with $q(x) > 0$ whenever $p(x) > 0$. The Kullback Leibler divergence is

$$D_{\text{KL}}(p\|q) = \int p(x) \log \frac{p(x)}{q(x)} dx = \mathbb{E}_p[\log p(X) - \log q(X)].$$

For probability mass functions $p(x)$ and $q(x)$ describing discrete random variables, the Kullback–Leibler divergence is

$$D_{\text{KL}}(p\|q) = \sum_{x \in \mathcal{X}} p(x) \log \frac{p(x)}{q(x)}.$$

Exercise 8.3.1

D_{KL} for either continuous or discrete random variables is not symmetric and does not satisfy triangle inequality. Show that this is true by considering the case of 2 Bernoulli distributions.

The non-negativity of D_{KL} follows from Jensen inequality (see Section 8.8.2). Since the function $-\log t$ for $t \in \mathbb{R}_{>0}$ is convex,

$$D_{\text{KL}}(p\|q) = \mathbb{E}_p \left[-\log \frac{q(X)}{p(X)} \right] \geq -\log \mathbb{E}_p \left[\frac{q(X)}{p(X)} \right] = -\log \int q(x) dx = 0.$$

One of the few cases in which the KL divergence can be evaluated analytically, is in the case of multivariate Gaussians. We'll rely on the below formula when we look at variational auto encoders and also diffusion models.

D_{KL} between two multivariate Gaussians

Suppose $p(x) = \mathcal{N}(x; \mu_0, \Sigma_0)$, and $q(x) = \mathcal{N}(x; \mu_1, \Sigma_1)$ with $x \in \mathbb{R}^d$, then

$$D_{\text{KL}}(p||q) = \frac{1}{2} \left[\log \frac{\det \Sigma_1}{\det \Sigma_0} - d + \text{tr}(\Sigma_1^{-1} \Sigma_0) + (\mu_1 - \mu_0)^\top \Sigma_1^{-1} (\mu_1 - \mu_0) \right].$$

It turns out that D_{KL} is closely related to the cross-entropy, hence an alternative name for D_{KL} is the *relative entropy*. The cross entropy of p relative to q is

$$H(p, q) = -\mathbb{E}_p[\log q(X)].$$

The entropy of p is

$$H(p) = -\mathbb{E}_p[\log p(X)].$$

Therefore

$$D_{\text{KL}}(p||q) = \mathbb{E}_p[\log p(X) - \log q(X)] = H(p, q) - H(p).$$

When used as an objective function in optimisation/model fitting, when p is fixed, minimising cross entropy in q is equivalent to minimising $D_{\text{KL}}(p||q)$. Furthermore, notice that $D_{\text{KL}}(p||q)$ requires evaluating an expectation with respect to $p(x)$. Being able to efficiently evaluate an objective function (or estimate it) is extremely important in machine learning. This sometimes dictates the direction (forward or reverse) that is adopted, as we will see later in Section 8.6. The following definitions are often used in the machine learning literature when p is the **target** or data distribution and q is the **approximation**,

- **Forward KL:**

$$D_{\text{KL}}(p||q) = \mathbb{E}_p \left[\log \frac{p(X)}{q(X)} \right].$$

Notice that the expectation is taken under p , so regions having substantial probability under p contribute more to the objective. This means that q is strongly penalised for failing to cover a mode of p . It is therefore often described as *mass-covering*.

- **Reverse KL:**

$$D_{\text{KL}}(q||p) = \mathbb{E}_q \left[\log \frac{q(X)}{p(X)} \right].$$

Since the expectation is taken under q , regions to which q assigns negligible probability contribute little, even if p assigns them substantial probability. The approximation may therefore concentrate on one prominent mode of p , and the reverse KL is often described as *mode-seeking*.

The KL divergence can behave poorly when the supports of the distributions $p(x)$ and $q(x)$ do not overlap. Thus the KL divergence can be unsuitable when comparing empirical distributions with disjoint or nearly disjoint support.

Exercise 8.3.2

Suppose there exists x^* where $p(x^*) > 0$ and $q(x^*) = 0$. What does this mean for $D_{\text{KL}}(p||q)$?

8.3.2 Fisher Divergence

The next important divergence we'll consider is the Fisher divergence which compares distributions through their score functions, i.e. the gradient of the log density.

Fisher Divergence

Let p and q be strictly positive, differentiable probability densities on a common domain in $\mathbb{R}^d$, and suppose that the following expectation is finite. The Fisher divergence from p to q is

$$D_F(p||q) = \frac{1}{2} \int \|\nabla_x \log p(x) - \nabla_x \log q(x)\|^2 p(x) dx = \frac{1}{2} \mathbb{E}_p \left[\|\nabla_x \log p(x) - \nabla_x \log q(x)\|^2 \right]$$

While D_{KL} compares log density values (and weights them where the baseline density is large), the Fisher divergence compares local derivatives of log densities. Hence D_F is sensitive to the local geometry of the distributions, rather than directly to probability mass mismatch. Loosely speaking, D_F compares (pointwise) the slope of two density functions whilst D_{KL} compares density function heights (pointwise). This makes it especially important in score matching and diffusion models, where one often learns $\nabla_x \log p(x)$ rather than the normalised density $p(x)$ itself (see Week 9 and 10 lectures).

As with D_{KL} , the Fisher divergence can be evaluated analytically in the case of multivariate Gaussians.

D_F between two multivariate Gaussians

Suppose

$$p(x) = \mathcal{N}(x; \mu_0, \Sigma_0), \quad q(x) = \mathcal{N}(x; \mu_1, \Sigma_1),$$

where $\Sigma_0, \Sigma_1 \in \mathbb{R}^{d \times d}$ are symmetric positive-definite covariance matrices. Then

$$D_F(p||q) = \frac{1}{2} \left[\text{tr}(\Sigma_1^{-2} \Sigma_0 - 2\Sigma_1^{-1} + \Sigma_0^{-1}) + (\mu_0 - \mu_1)^\top \Sigma_1^{-2} (\mu_0 - \mu_1) \right].$$

Exercise 8.3.3

We saw that D_{KL} has definitions both for continuous and discrete random variables. Why is the Fisher Divergence difficult to define for discrete random variables?

Exercise 8.3.4

Comparing Fisher and KL divergence. Consider the centred one-dimensional Gaussian distributions

$$p(x) = \mathcal{N}(0, 1), \quad q_\sigma(x) = \mathcal{N}(0, \sigma^2), \quad \sigma^2 > 0.$$

(a) Show that

$$D_{\text{KL}}(p \| q_\sigma) = \frac{1}{2} \left(\log \sigma^2 + \frac{1}{\sigma^2} - 1 \right).$$

(b) Compute the score functions of p and q_σ , and hence show that

$$D_{\text{F}}(p \| q_\sigma) = \frac{1}{2} \left(1 - \frac{1}{\sigma^2} \right)^2.$$

(c) Evaluate both divergences when $\sigma^2 = 100$ and $\sigma^2 = 0.01$.

(d) Explain what these calculations demonstrate about the ways in which the two divergences respond to an approximation that is much broader or much narrower than p .

8.4 Recap: Maximum Likelihood Estimation

We'll now have a brief recap of maximum likelihood estimation, one of the most important methods in frequentist statistics, and arguably one of the earliest forms of generative modelling. It is the standard method for fitting parametric distributions to observed data, where the goal is to find the parameter set that maximises the likelihood function.

Given some data, we may assume that $X_1, X_2, \dots, X_n \sim p_\theta(x)$. Our assumption here is that $p_\theta(x)$ is the correct model for this data, and only the parameters θ are unknown. The joint probability density of the observed data is

$$p_\theta(x_{1:n}) := p_\theta(x_1, \dots, x_n) = \prod_{i=1}^n p_\theta(x_i). \quad (8.3)$$

This quantity is viewed as a function of θ and is called the *likelihood function*, because it is not strictly speaking a probability density (or mass function) in θ . We will use the following notation for the likelihood

$$L(\theta) = \prod_{i=1}^n p_\theta(x_i). \quad (8.4)$$

The maximum likelihood estimator (MLE) is defined as the parameter value that maximises the likelihood,

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta \in \Theta} L(\theta). \quad (8.5)$$

It is often much easier to work with the log-likelihood. Since the logarithm is monotone increasing, maximising the likelihood is equivalent to maximising the log-likelihood,

$$\ell(\theta) := \log L(\theta) = \sum_{i=1}^n \log p_{\theta}(x_i). \quad (8.6)$$

Hence the MLE can also be written as

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta \in \Theta} \sum_{i=1}^n \log_{\theta} p(x_i). \quad (8.7)$$

The MLE selects the model parameter that assigns the highest probability density to the observed data. Equivalently, it chooses the density $p_{\theta}(x)$ that best fits the empirical distribution of the samples.

Finally, as we will see in Section 8.6, maximising the (expected) log-likelihood is equivalent to minimising D_{KL} between the true data generating process and the selected the parametric model.

8.4.1 The MLE for the exponential family

Recall that a probability density or mass function belongs to the exponential family if it can be written in the form

$$p_{\theta}(x) = b(x) \exp\left(\theta^{\top} t(x) - a(\theta)\right),$$

where

- $t(x) \in \mathbb{R}^m$ is the sufficient statistic;
- $\theta \in \mathbb{R}^m$ is the natural parameter;
- $b(x)$ is the base-measure term; and
- $a(\theta)$ is the log-partition function.

In the continuous case, the log-partition function is

$$a(\theta) = \log \int b(x) \exp\left(\theta^{\top} t(x)\right) dx,$$

while in the discrete case the integral is replaced by a sum. By construction, $a(\theta)$ ensures that p_{θ} integrates or sums to one.

Suppose that $X_1, \dots, X_n \stackrel{\text{i.i.d.}}{\sim} p_{\theta}$. Define the aggregated sufficient statistic

$$T(x_{1:n}) = \sum_{i=1}^n t(x_i)$$

and the log-likelihood $\ell(\theta)$ is

$$\ell(\theta) = \sum_{i=1}^n \log p_{\theta}(x_i) = \sum_{i=1}^n \log b(x_i) + \theta^{\top} T(x_{1:n}) - na(\theta).$$

Differentiating with respect to θ and setting to zero gives

$$\nabla_{\theta} \ell(\theta) = T(x_{1:n}) - n \nabla_{\theta} a(\theta) = 0.$$

Therefore, the maximum likelihood estimate satisfies

$$\frac{1}{n} \sum_{i=1}^n t(x_i) = \nabla_{\theta} a(\hat{\theta}_{\text{ML}}).$$

Exercise 8.4.1

Show that $\nabla_{\theta} a(\theta) = \mathbb{E}_{p_{\theta}}[t(X)]$.

Proof: We will assume differentiation may be interchanged with integration. Then

$$\begin{aligned} \nabla_{\theta} a(\theta) &= \nabla_{\theta} \log \int b(x) \exp(\theta^{\top} t(x)) \, dx \\ &= \frac{\int t(x) b(x) \exp(\theta^{\top} t(x)) \, dx}{\int b(x) \exp(\theta^{\top} t(x)) \, dx} \\ &= \int t(x) b(x) \exp(\theta^{\top} t(x) - a(\theta)) \, dx \\ &= \int t(x) p_{\theta}(x) \, dx \\ &= \mathbb{E}_{p_{\theta}}[t(X)]. \end{aligned}$$

■

Therefore, the MLE satisfies the moment-matching equation

$$\frac{1}{n} \sum_{i=1}^n t(x_i) = \mathbb{E}_{p_{\hat{\theta}_{\text{ML}}}}[t(X)].$$

The maximum likelihood method selects the parameter for which the expected sufficient statistic under the fitted model matches the corresponding empirical sufficient statistic.

Exercise 8.4.2

MLE for the Bernoulli distribution. Let

$$X_1, \dots, X_n \stackrel{\text{i.i.d.}}{\sim} \text{Bernoulli}(p), \quad 0 < p < 1.$$

The probability mass function is

$$p_p(x) = p^x(1-p)^{1-x}, \quad x \in \{0, 1\}.$$

- (a) Write down the likelihood and log-likelihood functions.
- (b) Differentiate the log-likelihood with respect to p .
- (c) Assuming that $0 < \sum_{i=1}^n x_i < n$, show that the maximum likelihood estimator is

$$\hat{p}_{\text{ML}} = \frac{1}{n} \sum_{i=1}^n x_i.$$

i.e. the sample mean.

Exercise 8.4.3

MLE for the Gaussian distribution. Let

$$X_1, \dots, X_n \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(\mu, \sigma^2), \quad \mu \in \mathbb{R}, \quad \sigma^2 > 0.$$

- (a) Write down the log-likelihood function for (μ, σ^2) .
- (b) Show that, for fixed σ^2 , the maximum likelihood estimator of μ is

$$\hat{\mu}_{\text{ML}} = \frac{1}{n} \sum_{i=1}^n x_i.$$

- (c) Show that the maximum likelihood estimator of the variance is

$$\hat{\sigma}_{\text{ML}}^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{\mu}_{\text{ML}})^2.$$

Remark 8.4.2

Then normalising constant (or log-partition function $a(\theta)$ for exponential family) is essential in maximum likelihood estimation. Since it depends on θ , it must be included when taking derivatives. Dealing with $a(\theta)$ is manageable for simple exponential families, but can be prohibitive in neural network models. We will see in next week's lectures how we can avoid dealing with a normalising constant.

8.4.2 Stochastic gradient ascent

When the MLE is not available analytically, it is common to rely on *stochastic gradient ascent* to obtain a suboptimal approximation (it converges to a local maximum of the likelihood). This proceeds in much the same way as you have already seen for stochastic gradient descent based methods for training neural networks.

Since evaluating the full gradient over all n observations can be expensive, stochastic gradient ascent approximates the gradient using a single randomly selected observation (or, more commonly, a small mini-batch).

The parameter update is

$$\theta_{t+1} = \theta_t + \eta_t \nabla_{\theta} \log p_{\theta_t}(x_i),$$

where x_i is sampled uniformly from the dataset and η_t is the learning rate.

More generally, using a mini-batch B_t ,

$$\theta_{t+1} = \theta_t + \eta_t \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_{\theta} \log p_{\theta_t}(x_i).$$

8.5 Latent Variable modelling

In Section 8.4, we saw how classical maximum likelihood estimation works when the data generating process $p_{\theta}(x)$ is known up to its parameters. In many cases, particularly with multi-modal, high dimensional data, modelling $p_{\theta}(x)$ directly can be difficult. Observations often exhibit structure that is more naturally explained through hidden or unobserved factors. For example,

- An **image** of a face may be influenced by identity, pose, lighting, expression, and camera viewpoint.
- A **text** sequence may be influenced by topic, style, and sentiment.
- A **speech** signal may be influenced by phonetic content, speaker identity, and background noise.

These factors are not usually observed directly in the dataset, but they help explain regularities in the observed data. Latent variable models (LVMs) are used to describe such hidden structure, reduce dimensionality, and explain dependencies among observed variables.

A latent (or hidden) variable z is a variable that is part of the probabilistic model but is not observed in the dataset and maybe continuous or discrete. In this section we will (mostly) assume $z \in \mathbb{R}^m$. Instead of modelling the observed variable x directly, a joint distribution over both observed and latent variables forms the probabilistic model.

A common latent variable model (LVM) has the factorisation

$$p_{\Theta}(x, z) = p_{\vartheta}(z) p_{\theta}(x|z),$$

which can be obtained directly from the definition of conditional probability and $\Theta = \theta \cup \vartheta$ or $\Theta = [\theta, \vartheta]^\top$ in the case of continuous z, x denotes all parameters of the model. This factorisation can be interpreted as the following generative process,

$$\begin{aligned} \text{Step 1. } & z^i \sim p_\vartheta(z), \\ \text{Step 2. } & x^i \sim p_\theta(x|z^i) \end{aligned}$$

The distribution $p_\vartheta(z)$ specifies how latent variables are generated (sometimes referred to as a *prior*), while $p_\theta(x|z)$ specifies how observations are generated given the latent variable. The resulting marginal distribution over observations is

$$p_\Theta(x) = \int p_\vartheta(z)p_\theta(x|z) dz,$$

or, if z is discrete,

$$p_\Theta(x) = \sum_z p_\vartheta(z)p_\theta(x|z).$$

Definition 8.5.1

In the variational inference machine learning literature, the quantity $p_\Theta(x)$ is called the *marginal likelihood* or *model evidence* and $p_\theta(x, z)$ is the *generative model*. You may find that in some modern generative modelling contexts, $p_\theta(x)$ is referred to as the generative model.

This construction can be highly expressive. Even if $p_\vartheta(z)$ and $p_\theta(x|z)$ are individually simple, the marginal distribution $p_\Theta(x)$ may be complicated because it averages over many possible latent explanations of the same observation. We will see a prototypical example of this below.

Example LVM: Gaussian mixture

A simple and important example of a LVM is the Gaussian mixture model. Suppose each observation $x \in \mathbb{R}^d$ belongs to one of K unobserved clusters. Let

$$z \in \{1, \dots, K\}$$

be a discrete latent variable indicating the cluster assignment. The latent variable is a categorical (or multinomial) random variable with

$$p(z = k) = \pi_k,$$

where $\pi_k \geq 0$, $\sum_{k=1}^K \pi_k = 1$, are the probabilities associated to the k th cluster and

$$p_\theta(x|z = k) = \mathcal{N}(x; \mu_k, \Sigma_k).$$

The joint distribution is therefore

$$p_\theta(x, z = k) = p(z)p_\theta(x|z = k) = \pi_k \mathcal{N}(x; \mu_k, \Sigma_k).$$

with $\theta = \{\mu_1, \Sigma_1, \mu_2, \Sigma_2, \dots, \mu_K, \Sigma_K\}$. Marginalising over the latent cluster assignment gives

$$p_\theta(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x; \mu_k, \Sigma_k).$$

yielding the familiar Gaussian mixture model. This example illustrates an important role of latent variables: they allow a complicated distribution to be represented as a combination of simpler conditional distributions. The latent variable z introduces hidden structure into the model, here corresponding to cluster membership.

Another setting where latent variables arise is in *Factor Analysis*, which is used to describe variability in high-dimensional observations using a smaller number of unobserved variables, called latent factors. Correlations between the observed variables are assumed to arise because they depend on the same latent variables. If the number of latent factors is much smaller than the number of observed variables, then the model provides a compact, low-dimensional description of the variability in the data.

Example LVM: Factor Analysis

Suppose we may model observed data $x \in \mathbb{R}^d$ as

$$\begin{aligned} x &= Wz + \mu + \epsilon, \\ z &\sim \mathcal{N}(0, I), \quad \epsilon \sim \mathcal{N}(0, \Psi). \end{aligned}$$

and z and ϵ are independent. Here $z \in \mathbb{R}^m$, with $m < d$, is a low-dimensional latent factor that explains correlations among the components of x . The matrix W maps latent factors into the observation space, while ϵ captures residual noise. The prior here is $p(z) = \mathcal{N}(0, I)$ and notice that the prior parameters are entirely specified (no estimation required, hence we use $p(z)$ rather than $p_\vartheta(z)$). Conditioned on the latent factors,

$$p_\theta(x|z) = \mathcal{N}(x; Wz + \mu, \Psi).$$

It then holds that

$$p_\theta(x) = \int p_\theta(x|z)p(z)dz = \mathcal{N}(x; \mu, WW^\top + \Psi),$$

(which can also be obtained by noticing that x is an affine transformation of Gaussian random variables). The covariance of the observed variable x is given by $\text{Cov}(x) = WW^\top + \Psi$ and $WW^\top \in \mathbb{R}^{d \times d}$ however $\text{rank}(WW^\top) = \text{rank}(W) \leq m < d$. The low-rank term $WW^\top$ captures correlations induced by the shared latent factors, while Ψ accounts for residual noise. Rather than modelling an arbitrary covariance matrix directly, factor analysis explains the covariance structure through a much smaller number of latent variables.

In [2], the term deep latent variable models is used to refer to the fact that the distributions themselves are parameterised by neural networks. We will see an example of this in next week's lecture on variational autoencoders.

Other examples where latent variables arise include hidden Markov models, where latent states explain sequential observations; topic models, where latent topics explain word co-occurrence patterns in documents; state-space models, where latent dynamical states explain observed time series data; and modern deep generative models, where latent codes are transformed by neural networks into observations such as images or audio.

8.5.1 Inference and learning in LVMs

Previously we saw the generative process of a fully specified LVM model. It is sometimes useful to instead focus on the *inference* process, the reverse process of generation,

$$x \longrightarrow z, \quad (\text{inference})$$

The inferential process asks which latent variable values likely generated this observation? This is often answered by invoking Bayes theorem,

$$p_{\Theta}(z|x) = \frac{p_{\Theta}(x, z)}{p_{\Theta}(x)} = \frac{p_{\vartheta}(z)p_{\theta}(x|z)}{\int p_{\vartheta}(z)p_{\theta}(x|z) dz}.$$

Definition 8.5.2

In the machine learning literature, $p_{\Theta}(z|x)$ is usually referred to as the *recognition model* but sometimes also referred to as the posterior (after Bayesian terminology).

In simple LVMs such as some Gaussian mixture models or factor analysis models, $p_{\Theta}(z|x)$ may be tractable. In more complex models, especially when $p_{\Theta}(z|x)$ is parameterised by a neural network, the posterior is usually intractable. This intractability motivates approximate inference methods, including variational inference (see Section 8.6).

Remark 8.5.1

The distribution $p_{\vartheta}(z)$ is sometimes referred to as the prior, and inference about the latent variables is at least conceptually very similar to computing the posterior distribution over parameters in Bayesian inference (see (8.2)). However, there are some differences between latent variable modelling and Bayesian modelling. For example, the parameters of the LVM ϑ, θ may be estimated in a frequentist way (e.g. via maximum likelihood estimation) even though the latent variable z is treated as a “Bayesian random variable,” for which we have prior knowledge quantified by $p_{\vartheta}(z)$.

Furthermore, in the machine learning context, estimating z given x is known as *inference*, whereas fitting the parameters Θ is known as *learning*. This terminology is more natural in the ML context, particularly when Θ are parameters of a neural network model which are traditionally ‘learnt’ using e.g. backpropagation. In classical statistics, the term *inference* is often used to refer to fitting the parameters of a statistical model.

Finally, latent variable models play two complementary roles:

1. They define flexible probability distributions over observed data by marginalising over hidden variables.
2. Second, they provide a framework for learning structured representations that summarise the underlying factors of variation in the data (which are usually not directly observed).

This viewpoint will be crucial when we get to Variational Auto Encoders in Week 9.

8.6 Variational Inference (VI)

Markov Chain Monte Carlo (MCMC) sampling is one of the most well established methods for Bayesian sampling, although even these methods struggle in the high dimensional settings encountered in many machine learning applications. MCMC methods are typically *exact*, in the sense that they are designed to theoretically generate a sample from π , but they can be computationally expensive to implement. We'll use π from now onwards to denote the generic target distribution of interest (to avoid confusion with $p(z)$ for the prior).

Variational inference (VI) emerged as an *inexact* alternative to make inference (and sampling) in high dimensions practically feasible. We will mainly discuss it in the context of sampling from a distribution on continuous random variables, although it is equally applicable to distributions over discrete random variables. Examples include Bayesian mixture models (where the mixture component is discrete) and topic models such as Latent Dirichlet Allocation (LDA). In fact, many of the earliest successes of variational inference were in such discrete latent variable models.

The main idea behind VI is to recast the sampling problem into an optimisation problem.

In VI, the aim is to solve the minimisation problem

$$q^* = \arg \min_{q \in \mathcal{Q}} D(q || \pi),$$

where

- π is the density to be sampled from (so-called *target*)
- $D(q, \pi)$ is a statistical distance between another density q and p
- $\mathcal{Q}$ is the so-called *variational family*, that is, a set or class of probability densities, to be chosen by the user.

VI methods are sometimes referred to as those that specifically choose $\mathcal{Q}$ to be a restricted class.⁴ If $\mathcal{Q}$ corresponded to all possible probability density functions on $\mathbb{R}^d$, this would be a rather large search space, making the optimisation problem infeasible in general. Therefore, choosing $\mathcal{Q}$ appropriately is a delicate task: it must be large enough to capture the essential features of p , but not so large that the optimisation problem would be intractable. However, there are certain cases in which it is possible to derive practically implementable methods even with $\mathcal{Q}$ being the space of all probability density functions on $\mathbb{R}^d$ (see gradient flows, out of scope of this course).

Exercise 8.6.1

What do you think would be some standard candidates for the variational family $\mathcal{Q}$?

⁴Sometimes the term ‘variational’ is also used to refer to methods where $\mathcal{Q}$ is taken to be the space of all probability densities, in which case $q^* = p$. In that case, the term ‘variational’ is used to refer to the ‘optimisation’ nature of the problem (coming from variational calculus). We will not consider these types of methods.

Remark 8.6.2

Assuming q is absolutely continuous with respect to π (i.e. integral is well-defined) and $\mathcal{Q}$ is the space of all such probability densities, there is a unique q such that $D_{\text{KL}} = 0$. Since $D_{\text{KL}} \geq 0$, this means the minimisation problem has a unique solution with $q^* = \pi$. When $\mathcal{Q}$ is restricted to some smaller space (e.g. space of Gaussian densities), then there is no guarantee in general that the minimisation problem has a unique solution, and its solution cannot equal π unless $\pi \in \mathcal{Q}$.

Remark 8.6.3

Recall from Section 8.3.1 that the forward and reverse KL divergences often produce qualitatively different approximations. Minimising the reverse KL is described as *mode seeking* since the approximation tends to concentrate its probability mass around one or a few high-density regions of the target distribution. In contrast, minimising the forward KL is generally *mass covering*, encouraging the approximation to place probability mass wherever the target distribution has support. Depending on the application, one behaviour may be preferable to the other.

8.6.1 Variational Bayes (VB)

The most prominent application of variational inference is in Bayesian inference or in inference of latent variable models, in fields ranging from computational neuroscience to robotics. To simplify presentation, we'll focus on inference in latent variable models, especially as this will be relevant for Week 9. Recall from Section 8.5.1 (dropping the parameter dependence),

$$p(z|x) = \frac{p(x|z)p(z)}{\int p(x|z)p(z)dz} \quad (8.8)$$

From here onwards, x could be either a single data point or iid data set. The target distribution here is the posterior, i.e. $\pi(z) = p(z|x)$. In VB, the statistical distance is almost always chosen to be the *reverse* Kullback–Leibler divergence⁵ $D_{\text{KL}}(q(z)||p(z|x))$, rather than the *forward* divergence, $D_{\text{KL}}(p(z|x)||q(z))$. The reverse KL divergence naturally gives rise to the so-called *Evidence Lower Bound* (ELBO), which yields an implementable objective function (see below).

Exercise 8.6.2

From a computational point of view, why would it be desirable to use the reverse rather than the forward KL?

From a variational inference point of view, approximating the posterior distribution is then a matter of solving

$$q^*(z) = \arg \min_{q \in \mathcal{Q}} D_{\text{KL}}(q(z)||p(z|x)), \quad (8.9)$$

⁵Other less common distances include the Jensen-Shannon divergence and Wasserstein distance

So far all we have done is replace a generic probability density π with a more structured density, the posterior. Evaluating this objective function requires having access to $p(z|x)$. Evaluating this using Bayes theorem requires evaluating the normalising constant $\int p(x|z)p(z)dz$, which can be high dimensional integral. We can avoid this and instead manipulate the objective function into a more tractable quantity, using the *Evidence Lower Bound Objective* (ELBO),

$$\text{ELBO}(q(z); x) := \mathbb{E}_{z \sim q}[\log p(x|z)] - \text{KL}(q(z)||p(z)). \quad (8.10)$$

We can manipulate the original objective function to obtain

$$\text{KL}(q(z)||p(z|x)) = \int \log \left(\frac{q(z)}{p(z|x)} \right) q(z) dz \quad (8.11)$$

$$= \int (\log q(z) - \log p(z|x)) q(z) dz \quad (8.12)$$

$$= \int (\log q(z) - \log p(z) - \log p(x|z) + \log p(x)) q(z) dz \quad (8.13)$$

$$= \int (\log q(z) - \log p(z)) q(z) dz - \int \log p(x|z) q(z) dz + \log p(x) \underbrace{\int q(z) dz}_{=1} \quad (8.14)$$

$$= \text{KL}(q(z)||p(z)) - \mathbb{E}_{z \sim q}[\log p(x|z)] + C \quad (8.15)$$

$$= -\text{ELBO}(q(z); x) + C \quad (8.16)$$

using the shorthand notation $p(x) = \int p(x|z)p(z)dz$, for the model evidence. As this object doesn't depend on z , it can be treated as a constant in the optimisation problem (hence why we label it C in the above). This means that (8.9) can be re-cast as a maximisation problem

$$q^*(z) = \arg \max_{q \in \mathcal{Q}} \text{ELBO}(q(z); x),$$

where the objective function is now easier to evaluate. Although we must still evaluate an expectation, it is an expectation with respect to $q(z)$ which can be approximated easily using Monte Carlo, since $q(z)$ is generally chosen so that it is easy to sample from.

Remark 8.6.4

Maximising the ELBO also has a nice interpretation: it balances finding a density $q(z)$ that optimises fit to the prior density $p(z)$ (by minimising $\text{KL}(q(z)||p(z))$) while also fitting to the data (by maximising the expected log-likelihood). This interpretation extends to the Bayes posterior $p(z|x)$, which is the unique optimiser of the ELBO (if $\mathcal{Q}$ contains it).

Exercise 8.6.3

The ELBO is named for the fact that it lower bounds the log evidence. Using the above derivation, show why this is true.

8.6.2 Mean Field VI

We now describe one of the simplest and most widely used choices of variational family, known as the *mean field variational family*. Consider the case $z \in \mathbb{R}^m$ where $z = [z_1, z_2, \dots, z_m]^\top$. The mean field approximation assumes that the components of z are mutually independent under the variational distribution. That is, a generic member of the mean field family has the factorised form

$$q(z) = \prod_{j=1}^m q_j(z_j),$$

where $q_j(z_j)$ is a separate variational distribution for the latent variable z_j , known as a *variational factor*. The optimisation problem is therefore to choose these factors so that the product distribution $q(z)$ is close to the true posterior $p(z|x)$ in reverse KL divergence.

The mean field assumption should not be interpreted as an assumption about the true posterior. The posterior $p(z|x)$ may contain strong dependencies between the latent variables. Rather, the independence assumption is imposed on the approximating family $\mathcal{Q}$ in order to make the optimisation problem tractable. By factorising the variational distribution, the optimisation problem often decomposes into simpler subproblems, and in many conjugate models the coordinate updates can be obtained in closed form.

Remark: Mean Field VI and uncertainty underestimation

The mean field family can be expressive in the sense that each marginal factor $q_j(z_j)$ may be chosen flexibly. However, by construction it cannot represent posterior correlations between different latent variables. This is one reason why mean field variational inference often underestimates posterior uncertainty, see for example Figure 8.2.

Another reason is closely related to the use of the reverse KL divergence, $\text{KL}(q(z)||p(z|x))$. The reverse KL penalises placing probability mass in regions where the true posterior has little mass, so that the fitted variational distribution often concentrates on a high-density region of the posterior. In practice, this means that mean field variational inference can give useful posterior means or point estimates, while underestimating uncertainty.

Remark 8.6.6

When approximating a Bayes posterior with n observations, $p(z|x_{1:n})$, classical mean field VI adopts a separate variational factor for each observation, i.e.

$$q_j(z_j) \approx p(z_j|x_j), \quad j = 1, 2, \dots, n$$

Since different observations generally induce different posterior distributions, each factor has its own variational parameters, which must be optimised separately. When a new observation x_{new} is encountered, there is no pre-existing variational factor for its latent variable, and a new optimisation problem must be solved to construct an approximation to $p(z_{\text{new}} | x_{\text{new}})$. This procedure is highly inefficient and so motivated the introduction of *amortised inference* in VAEs (more next week).

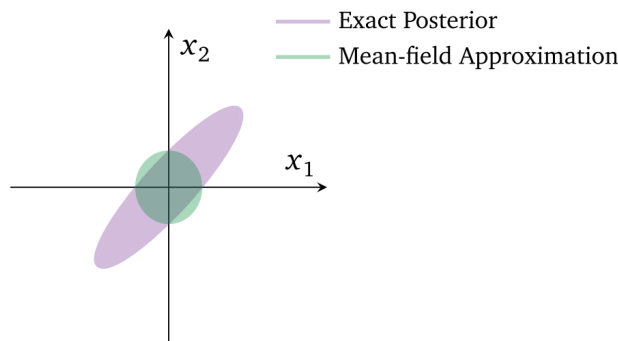


Figure 8.2: Example of mean-field VI with a Gaussian variational family and a 2D correlated Gaussian posterior. Shaded regions indicate 2 standard deviations from mean. (from [3])

Despite these limitations, mean field variational inference remains an important baseline method. It is simple, sometimes scalable, and often effective in large probabilistic models. More expressive variational families, such as structured variational approximations, relax the independence assumption by introducing dependencies between latent variables, but this typically leads to a more difficult optimisation problem.

8.6.3 [NON-EXAMINABLE] Coordinate Ascent Variational Inference (CAVI)

We'll now demonstrate how one of the earliest methods of implementing mean field variational inference works, known as *Coordinate Ascent Variational Inference (CAVI)*. Recall that under the mean-field approximation, the variational distribution is factorised as

$$q(z) = \prod_{j=1}^m q_j(z_j).$$

The aim is to choose the factors $q_1, \dots, q_m$ to maximise the ELBO. Optimising all of the variational factors simultaneously is usually computationally expensive. CAVI instead updates one factor at a time. At each step, one factor $q_j(z_j)$ is optimised while all of the remaining factors are held fixed. The method cycles through the factors repeatedly until the ELBO no longer changes (up to some specified tolerance). In the below, we'll use the notation $\pi(z)$ to denote any target distribution (e.g. one example could be the bayes posterior $p(z|x)$).

The coordinate level updates are derived as follows. First consider

$$\begin{aligned} q(z) &= q_j(z_j)q_{-j}(z_{-j}) \\ q_{-j}(z_{-j}) &= \prod_{i \neq j} q_i(z_i), \end{aligned}$$

where z_{-j} denotes all components of z except z_j . The ELBO here is

$$\text{ELBO}(q) = \mathbb{E}_q[\log \pi(z)] - \mathbb{E}_q[\log q(z)].$$

When all factors except q_j are held fixed, this can be written as

$$\text{ELBO}(q) = \mathbb{E}_{q_j} [\mathbb{E}_{q_{-j}} [\log \pi(z)]] - \mathbb{E}_{q_j} [\log q_j(z_j)] + C,$$

where C contains all terms that do not depend on q_j .

Define

$$\tilde{p}_j(z_j) \propto \exp \left\{ \mathbb{E}_{q_{-j}} [\log \pi(z)] \right\}.$$

Substituting this into (8.6.3) yields that the terms in the ELBO that depend on q_j can be expressed as

$$-D_{\text{KL}}(q_j(z_j) \parallel \tilde{p}_j(z_j)) + C_1,$$

for some constant C_1 , meaning that the ELBO is maximised when

$$q_j^*(z_j) = \tilde{p}_j(z_j).$$

Hence the general CAVI update is

$$q_j^*(z_j) \propto \exp \left\{ \mathbb{E}_{q_{-j}} [\log p(z)] \right\},$$

or equivalently,

$$\log q_j^*(z_j) = \mathbb{E}_{q_{-j}} [\log p(z)] + C_2 \quad (8.17)$$

where the expectation is taken over all variables except z_j , and the additive constant C_2 does not depend on z_j .

Thus, each CAVI step consists of taking the expected log target density with respect to the other variational factors, exponentiating the result, and normalising it to obtain an updated probability distribution for z_j .

8.6.3.1 Illustrative Example: Correlated Gaussian target.

We'll look at a highly simplified setting where the target distribution (or Bayes posterior) is a 2D correlated Gaussian. Let $z = [z_1, z_2]^\top$ and

$$\pi(z) = \mathcal{N} \left(z; 0, \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix} \right), \quad |\rho| < 1$$

be the target distribution. The two components are correlated whenever $\rho \neq 0$. We approximate π using the mean-field variational family

$$\mathcal{Q} = \{q(z) = q_1(z_1)q_2(z_2)\}$$

where $q_j(z_j)$ is a univariate Gaussian. We'll now calculate all the necessary identities to apply CAVI to this problem.

Log density of the target. The inverse covariance matrix is

$$\Sigma^{-1} = \frac{1}{1-\rho^2} \begin{pmatrix} 1 & -\rho \\ -\rho & 1 \end{pmatrix}.$$

Therefore, up to an additive constant,

$$\log \pi(z_1, z_2) = -\frac{1}{2(1-\rho^2)} (z_1^2 - 2\rho z_1 z_2 + z_2^2).$$

Update for $q_j(z_j)$. Applying the general coordinate update (8.17), we have

$$\log q_1^*(z_1) = -\frac{1}{2(1-\rho^2)} (z_1^2 - 2\rho m_2 z_1) + C$$

where $m_2 = \mathbb{E}_{q_2}[z_2]$ and any terms independent of z_1 are pulled into the constant. Completing the square gives

$$q_1^*(z_1) = \mathcal{N}(z_1; \rho m_2, 1 - \rho^2).$$

By a similar calculation for q_2 , we obtain

$$q_2^*(z_2) = \mathcal{N}(z_2; \rho m_1, 1 - \rho^2).$$

where $m_1 = \mathbb{E}_{q_1}[z_1]$. So then the CAVI mean updates are

$$m_1 \leftarrow \rho m_2, \quad m_2 \leftarrow \rho m_1,$$

while the variance updates are fixed at $1 - \rho^2$ and do not need updating. This is a result of the simplicity of this example, in general, all the parameters of the variational factors would need updating.

Coordinate ascent algorithm. Starting from initial means $m_1^{(0)}$ and $m_2^{(0)}$, CAVI alternates the updates

$$\begin{aligned} m_1^{(t+1)} &= \rho m_2^{(t)}, \\ m_2^{(t+1)} &= \rho m_1^{(t+1)}. \end{aligned}$$

Because $|\rho| < 1$, the means converge to

$$m_1^* = m_2^* = 0.$$

Therefore the optimal mean-field approximation is

$$\begin{aligned} q^*(z) &= \mathcal{N}(z_1; 0, 1 - \rho^2) \mathcal{N}(z_2; 0, 1 - \rho^2) \\ &= \mathcal{N}\left(z; \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 - \rho^2 & 0 \\ 0 & 1 - \rho^2 \end{bmatrix}\right). \end{aligned}$$

Notice that π has $\text{Cov}_p(z_1, z_2) = \rho$ but by the mean field assumption, the approximation has $\text{Cov}_p(z_1, z_2) = 0$. It therefore cannot reproduce the orientation of the target density. Moreover, the marginal variances of the target are 1, whereas the variational approximation has

$$\text{Var}_{q^*}(z_1) = \text{Var}_{q^*}(z_2) = 1 - \rho^2 < 1$$

whenever $\rho \neq 0$. This is just one concrete example of how the mean-field reverse-KL approximation underestimates the marginal uncertainty. It concentrates inside the high-density region of the target rather than placing probability mass in the lower-density regions required to reproduce the full marginal variances, similar to Figure 8.2.

8.7 Tutorial exercises

Exercise 1. Complete all exercises in the notes.

Exercise 2. Suppose that $Z \in \mathbb{R}$ is a continuous random variable with probability density function $p_Z(z)$, and let

$$U = g(Z)$$

be a continuously differentiable, strictly monotonic transformation.

The change-of-variable formula states that

$$p_U(u) = p_Z(g^{-1}(u)) \left| \frac{d}{du} g^{-1}(u) \right|.$$

Denote the cumulative distribution function (CDF) of Z by

$$F_Z(z) := \mathbb{P}(Z \leq z).$$

- i. Suppose $U = F_Z$. Using the change-of-variable formula above, show that

$$U \sim \text{Uniform}(0, 1).$$

- ii. Hence show that if

$$U \sim \text{Uniform}(0, 1),$$

then

$$Z = F_Z^{-1}(U)$$

has density $p_Z(z)$. This result is known as *inverse transform sampling*

Exercise 3. Suppose we want to approximate a target distribution that is a two-dimensional Gaussian

$$\pi(z) = \mathcal{N}(z; \mu_p, \Sigma_p),$$

where

$$\mu_p = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \quad \Sigma_p = \begin{pmatrix} 1 & 0.8 \\ 0.8 & 1 \end{pmatrix}.$$

Consider the Gaussian variational approximation

$$q(z) = \mathcal{N}(z; \mu_q, \Sigma_q),$$

where

$$\mu_q = \begin{pmatrix} 0.5 \\ -0.5 \end{pmatrix}, \quad \Sigma_q = \begin{pmatrix} 0.6 & 0 \\ 0 & 0.6 \end{pmatrix}.$$

- (i) Use the analytic Gaussian formula to calculate the forward and reverse KL,

$$D_{\text{KL}}(q\|p) \quad \text{and} \quad D_{\text{KL}}(p\|q).$$

- (ii) Write down the form of a Monte Carlo estimator for both $D_{\text{KL}}(q\|p)$ and $D_{\text{KL}}(p\|q)$. Evaluate this estimator in your favourite programming language for

$$N \in \{100, 1000, 10000\}.$$

and compare it to the analytic expression in (i).

- (iii) Repeat the Monte Carlo calculations several times using different random seeds. Comment on how the variability of the estimator changes with N .

Exercise 4. Consider the one-dimensional bimodal target distribution

$$\pi(x) = \frac{1}{2}\mathcal{N}(x; -3, 1) + \frac{1}{2}\mathcal{N}(x; 3, 1).$$

Our goal is to approximate π via variational inference. The variational family will be the univariate Gaussian family,

$$q_{m,s}(x) = \mathcal{N}(x; m, s^2), \quad s > 0.$$

Unlike the KL divergence between two Gaussian distributions, the KL divergence between a Gaussian and a Gaussian mixture generally does not have a convenient closed-form expression. It can instead be estimated using Monte Carlo integration.

- (i) Write the reverse KL divergence as

$$D_{\text{KL}}(q_{m,s}\|\pi) = \mathbb{E}_{q_{m,s}} [\log q_{m,s}(x) - \log \pi(x)].$$

Hence construct the Monte Carlo estimator

$$\hat{D}_{\text{KL}}^{\text{rev}}(m, s) = \frac{1}{N} \sum_{i=1}^N [\log q_{m,s}(X^{(i)}) - \log \pi(X^{(i)})],$$

where

$$X^{(1)}, \dots, X^{(N)} \stackrel{\text{i.i.d.}}{\sim} q_{m,s}.$$

- (ii) Write the forward KL divergence as

$$D_{\text{KL}}(\pi\|q_{m,s}) = \mathbb{E}_{\pi} [\log \pi(x) - \log q_{m,s}(x)].$$

Describe how to draw samples from π , and hence construct a Monte Carlo estimator of the forward KL divergence.

- (iii) Using a large Monte Carlo sample, such as $N = 10,000$, estimate the reverse KL for each of the following variational distributions:

$$q_1(x) = \mathcal{N}(x; -3, 1),$$

$$q_2(x) = \mathcal{N}(x; 0, 10),$$

and

$$q_3(x) = \mathcal{N}(x; 3, 1).$$

Which distributions are preferred by the reverse KL? Explain the result.

- (iv) Estimate the forward KL for the same three candidate distributions. Which candidate is preferred by the forward KL? Explain why its behaviour differs from that of the reverse KL.
- (v) Show analytically that the Gaussian which minimises

$$D_{\text{KL}}(\pi \| q_{m,s})$$

must match the mean and variance of π . Hence find the optimal values of m and s^2 .

- (vi) Based on the preceding calculations, explain why reverse KL is described as mode seeking, whereas forward KL is described as mass covering.

8.8 Supplementary Material [NON-EXAMINABLE]

8.8.1 Multivariate Gaussian Random sampling

Example 8.2.1 is the two-dimensional special case of the standard method used to sample from a multivariate normal distribution. Let $Z \in \mathbb{R}^d$ and

$$Z \sim N(\mathbf{0}, I_d),$$

and let Σ be a positive-definite covariance matrix. If L satisfies

$$LL^\top = \Sigma$$

(for example, the Cholesky factor of Σ), then

$$X = \mu + LZ$$

has density

$$p_{\mathbf{X}}(x) = p_{\mathbf{Z}}(L^{-1}(x - \mu)) |\det(L^{-1})|,$$

which simplifies to the multivariate normal density $N(\mu, \Sigma)$. In practice, multivariate Gaussian sampling is typically performed exactly in this way: one first generates independent univariate standard Gaussian random variables and then applies an appropriate linear (or affine) transformation determined by the target covariance matrix.

8.8.2 Jensen's inequality

Let $\varphi : \mathbb{R} \rightarrow \mathbb{R}$ be a convex function and let X be a random variable for which the relevant expectations exist. Then

$$\varphi(\mathbb{E}[X]) \leq \mathbb{E}[\varphi(X)].$$

If φ is strictly convex, equality holds only when X is constant almost surely.

For a concave function, the direction of the inequality is reversed.

References

- [1] Tianhong Li, Dina Katabi, and Kaiming He. Return of unconditional generation: A self-supervised representation generation method. *arXiv preprint arXiv:2312.03701*, 2024. Version 3.
- [2] Diederik P Kingma and Max Welling. An introduction to variational autoencoders. *Foundations and Trends in Machine Learning*, 12(4):307–392, 2019.
- [3] David M. Blei, Alp Kucukelbir, and Jon D. McAuliffe. Variational Inference: A Review for Statisticians. *Journal of the American Statistical Association*, 112(518):859–877, April 2017. arXiv:1601.00670 [cs, stat].